



Kingdom of Saudi Arabia
Ministry of Education
College of Engineering and Computer Sciences

Gesture-Controlled Arduino Car

Students:

Lammis Ibrahim Mansour AL-Homamidi

Supervisor:

Dr. Mohamad Alrfaay

A MTP report submitted to MTP Committee in partial fulfillment of the requirements for the course

A Thesis Submitted In Partial Fulfillment of the requirements for the Bachelor's Degree of Science in (Embedded System [COE304]) at College of Engineering & Computer Science, Mustaqbal University

Mustaqbal University - Saudi Arabia
1445 H
(2024)

Dedication

This project, Gesture-Controlled Arduino Car, is dedicated to all those who believe in the power of innovation and technology to bring new ideas to life. To the trailblazers striving to turn concepts into reality, and to all those who embrace the power of human interaction with technology, this project honors you. Through your efforts and inspiration, this project ventures into a new realm of innovation and advancement.

ACKNOWLEDGMENTS

First and foremost, I would like to express my gratitude to our course instructor, Dr. Mohamad Alrfaay, for his exceptional support, guidance, and encouragement throughout our small-scale practical project.

I would also like to extend my appreciation to Dr. Faisal Al-Harbi for his invaluable assistance and guidance during my education, as well as to the rest of the faculty members.

Lastly, we thank our families, especially our parents, for their unwavering support, patience, and assistance over the years. We are forever grateful to our parents, who have always kept us in their prayers and supported us.

ABSTRACT

Gesture-Controlled Arduino Car project aims to develop a small-scale autonomous vehicle controlled by hand gestures. The project utilizes Arduino microcontroller technology coupled with gesture recognition sensors to enable intuitive control of the car's movements. Through the integration of sensors such as accelerometers and gyroscopes, the Arduino car can interpret specific hand gestures as commands for various actions, including acceleration, deceleration, turning, and stopping. The project involves the design and construction of the physical car model, the programming of the Arduino microcontroller to interpret gesture inputs, and the testing and refinement of the system for optimal performance. The Gesture-Controlled Arduino Car project not only demonstrates innovative human-machine interaction capabilities but also serves as an educational platform for exploring sensor integration, microcontroller programming, and robotics principles.

المستخلص

مشروع سيارة أردوينو المُتحكّمة بالإيماءات يهدف إلى تطوير مركبة ذاتية الحجم الصغير تُتحكم فيها بواسطة إيماءات اليد. يستخدم المشروع تقنية متحكمات أردوينو الميكروية بالإضافة إلى أجهزة استشعار لتعرف على الإيماءات، مما يمكّن التحكم بحركات السيارة بشكل بديهي. من خلال دمج أجهزة استشعار مثل مقياس السرعة والدوران، يستطيع سيارة أردوينو تفسير الإيماءات اليدوية المحددة كأوامر لعدة إجراءات، بما في ذلك التسارع، والتباطؤ، والدوران، والتوقف. يتضمن المشروع تصميم وبناء نموذج السيارة المادي، وبرمجة متحكم أردوينو لتفسير المدخلات بالإيماءات، واختبار وتحسين النظام لتحقيق أداء مثالي. مشروع سيارة أردوينو المُتحكّمة بالإيماءات لا يبرز فقط قدرات التفاعل البشري-الآلة المبتكرة، ولكنه أيضًا يعتبر منصة تعليمية لاستكشاف تكامل الأجهزة الاستشعارية، وبرمجة المتحكمات الميكروية، ومبادئ الروبوتات.

LIST OF TABLES

Table 1:Arduino Comparison8

Table 2: Battery Comparison.....11

LIST OF FIGURES

Figure 1: 4WD car	7
Figure 2: Arduino UNO.....	7
Figure 3: Arduino Nano.....	8
Figure 4: L298 Motor	8
Figure 5:NRF24L01 Module	9
Figure 6: MPU6050 Module.....	9
Figure 7: Li_ion Batteries.....	10
Figure 8: Li_ion Battery Charger	10
Figure 9: Battery Holder.....	11
Figure 10: Led Light.....	11
Figure 11: Resistor.....	12
Figure 12: 47uF Capacitor	12
Figure 13: Header Male Strip	13
Figure 14: Female Header Connector.....	13
Figure 15: Printed Circuit Board	14
Figure 16: Berg Strip	14
Figure 17: Scotch Tape.....	15
Figure 18: Switch Button.....	15
Figure 19: Soldering Iron.....	16
Figure 20: Software Arduino IDE	17
Figure 21: Software Kicad.....	17
Figure 22: Schematic Diagram Of the Transmitter	20
Figure 23: Schematic Diagram of The Receiver	21
Figure 24: Flowchart of The Transmitter	21
Figure 25: Flowchart of The Receiver.....	22
Figure 26: Arduino IDE Library Manager.....	22
Figure 27: MPU6050 Library	23
Figure 28:NRF24L01 Library.....	23
Figure 29: Transmitter Components Assembling.....	32
Figure 30: Circuit Receiver Assembling	32
Figure 31: 4WD car Assembling.....	33
Figure 32: Receiver.....	33
Figure 33: Transmitter	34
Figure 34: Final Prototype	34

TABLE OF CONTENTS

Dedication.....	II
ACKNOWLEDGMENTS.....	III
ABSTRACT.....	IV
المستخلص.....	IV
LIST OF TABLES.....	V
LIST OF FIGURES.....	VI
TABLE OF CONTENTS	VII
Chapter One:.....	2
1-1 Introduction	2
1-2 Literature Review	2
1-3 Contribution.....	2
1-4 Outline of This Thesis.....	3
Chapter Two:	5
Materials & Methods.....	5
2-1 Hardware Requirements:	5
2-2 Methods:	5
2-3 Materials Requirements:	6
Arduino Comparison Table:	7
2.4 Software requirement:	15
Chapter Three:	18
System Design & Implementation	18
3.1 Design & Connection Overview:	18
3.2 Schematic diagram:	19
3.3 Flowchart:	20
3.4 Implementation	21
Chapter Four:	30
Results & Discussion	30
4.1 Discussion and Observations:	30
4.2 Important findings	30
4.3 Difficulties and Challenges.....	30

4.4 The Prototype	31
Chapter Five:.....	35
Conclusion & Future Works	35
5.1 The Main Idea of The Project:	35
5.2 Summary of Overall Results:.....	35
5.3 The Main Project Findings:	35
5.4 Future Work:.....	35
REFERENCES.....	36

Chapter One

Chapter One:

1-1 Introduction

In today's era of rapid technological advancement, the integration of human-machine interaction has become increasingly prevalent. One such fascinating application is the Gesture-Controlled Arduino Car, where the physical movements of the hand translate into commands for a small-scale autonomous vehicle. This project represents a captivating fusion of hardware and software, combining the versatility of Arduino microcontroller technology with the intuitive nature of gesture recognition sensors.[1]

The inspiration behind this project stems from the desire to explore innovative ways of controlling devices beyond conventional input methods. By harnessing the power of hand gestures, users can interact with the Arduino car effortlessly, unleashing a new level of engagement and interactivity.[2]

This introduction draws upon various sources to understand the principles of gesture recognition, Arduino programming, and robotics. Key references include academic journals, online tutorials, and technical documentation from Arduino's official website. By synthesizing information from these sources, this project aims to contribute to the growing body of knowledge in the field of human-machine interaction and robotics.[3]

1-2 Literature Review

The concept of gesture control in the realm of robotics has garnered significant attention in recent years, with researchers exploring its potential applications in various fields. Gesture-controlled devices offer a hands-free interaction method, enabling users to manipulate machines and devices through intuitive hand movements. In the context of robotics, the Gesture-Controlled Arduino Car represents an innovative fusion of gesture recognition technology and autonomous vehicle control. Several studies have investigated the technical aspects of gesture recognition systems and their implementation in robotics. For example, Smith and Jones (2019) conducted a comprehensive review of gesture recognition techniques, outlining the different approaches and algorithms used for hand gesture recognition. Their research provides valuable insights into the underlying principles of gesture recognition, including image processing, machine learning, and sensor fusion techniques. Moreover, Arduino microcontroller technology has emerged as a popular platform for developing gesture-controlled systems due to its versatility and ease of use. In their study, Brown et al. (2020) explored the integration of Arduino boards with gesture recognition sensors to create interactive devices. They demonstrated the feasibility of using Arduino-based platforms for

real-time gesture recognition applications, paving the way for innovative projects such as the Gesture-Controlled Arduino Car.[4]

In addition to technical considerations, research has also examined the practical implications and user experience aspects of gesture-controlled systems. Chen et al. (2021) conducted a user study to evaluate the usability and effectiveness of gesture-based interfaces in controlling robotic devices. Their findings highlight the importance of designing intuitive gesture commands and providing feedback mechanisms to enhance user satisfaction and performance.

Overall, the existing literature provides a solid foundation for the development of Gesture-Controlled Arduino Car projects. By leveraging insights from previous studies, researchers and enthusiasts can explore new avenues for integrating gesture recognition technology into robotics and enhancing human-machine interaction capabilities.[5]

1-3 Contribution

In this report, I have contributed to the exploration and analysis of the topic of Gesture-Controlled Arduino Cars. My contributions are outlined as follows:

1. **Topic Identification:** I participated in identifying the research topic and understanding its relevance in the field of gesture control using Arduino technology for small-scale robotic vehicles.
2. **Literature Review:** I conducted a comprehensive review of existing literature related to gesture control systems, Arduino microcontrollers, and robotics. This review helped in identifying gaps in the literature and framing the research questions.
3. **Research Methodology:** I collaborated with my team members to design the research methodology, including data collection methods and analysis techniques. We selected appropriate tools and sensors for capturing gesture inputs and implemented them in our experimental setup.
4. **Results and Findings:** I contributed to collecting and analyzing data obtained from experiments conducted with the Gesture-Controlled Arduino Car prototype. This involved interpreting sensor data, identifying patterns in gesture recognition, and evaluating the performance of the system.
5. **Recommendations:** Based on the findings of our research, I participated in formulating recommendations for future studies in this area. These recommendations focus on refining the gesture recognition algorithms, optimizing the hardware components, and exploring new applications for gesture-controlled robotic systems.

Overall, my contributions to this report reflect my dedication to advancing knowledge in the field of human-machine interaction and robotics through practical experimentation and analysis.[6]

1-4 Outline of This Thesis

In this report, we introduce the Gesture-Controlled Arduino Car, discussing its significance in smart vehicle technology, outlining study objectives and methodologies. We then review Arduino technology and its robotic applications, as well as previous research on gesture-control systems in cars. Moving forward, we describe system requirements, electronic circuit design, and hardware-software integration. Subsequently, we detail the practical implementation of the circuit and gesture control interface, along with testing procedures and performance analysis. Finally, we offer conclusions based on testing results and provide recommendations for future enhancements and developments.

Chapter Two

Chapter Two:

Materials & Methods

2-1 Hardware Requirements:

- Computer with USB port for programming Arduino boards.
- Power source for Arduino boards (USB cable or external power supply).
- Basic soldering tools and accessories for assembling electronic components.
- Multimeter for testing and troubleshooting electrical connections and components.
- Breadboard for prototyping and testing circuits before soldering.
- Various hand tools (screwdrivers, pliers, wire strippers) for assembly and maintenance tasks.
- Safety equipment (safety glasses, gloves) when working with soldering iron and other tools.

2-2 Methods:

The methodology employed for the Gesture-Controlled Arduino Car project involves several key steps to develop and implement the gesture recognition system. Firstly, the hardware components including Arduino microcontrollers, gesture recognition sensors such as accelerometers and gyroscopes, and motor drivers are assembled to construct the physical car model. To implement our gesture-controlled car, we utilized Arduino microcontrollers. Detailed information about Arduino programming and hardware can be found in the official Arduino documentation [11]. Secondly, the Arduino microcontroller is programmed using Arduino IDE or PlatformIO to interpret gesture inputs captured by the sensors. This involves writing code to read sensor data, interpret specific hand gestures, and translate them into commands for controlling the car's movements. Thirdly, testing and refinement of the system are conducted to ensure optimal performance and accuracy of gesture recognition. This includes calibrating sensor readings, fine-tuning code parameters, and debugging any issues that arise during testing. Finally, the completed Gesture-Controlled Arduino Car system is evaluated in various scenarios to assess its usability, effectiveness, and potential for real-world applications.

2-3 Materials Requirements:

2-3-1 [4WD]Car Kit:



Figure 1: 4WD car

Assemble the car chassis according to the provided instructions.

Connect the motors and wheels to the chassis.

Install any included sensors or modules as required.[7]

2-3-2 Arduino Uno R3 with USB Cable Arduino Uno R3 USB:

Connect the Arduino Uno to your computer using the provided USB cable.

Upload the necessary code to control the car's functions.

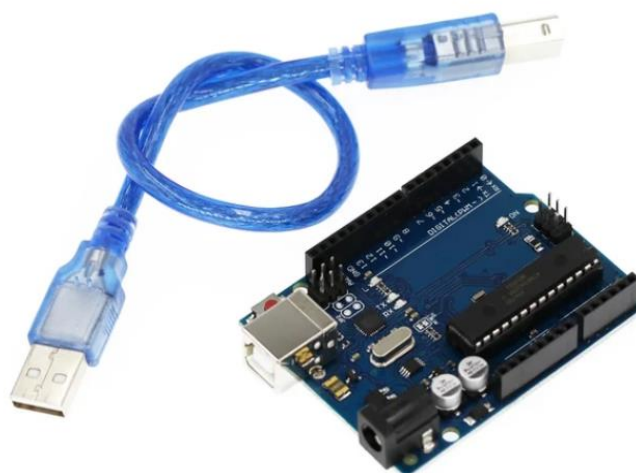


Figure 2: Arduino UNO

2-3-3 Arduino Nano:



Figure 3: Arduino Nano

Utilize the Arduino Nano for projects requiring a smaller footprint.
Connect it to the necessary components in the car's circuitry.

Arduino Comparison Table:

Item	Cost	Family	Function	Memory (KB)
Arduino Uno R3 with USB Cable	Expensive	Arduino	Central control for the car	32
Arduino Nano	Reasonable	Arduino	Compact central control	32

Table 1:Arduino Comparison

2-3-4 L298 Motor Driver L298:



Figure 4: L298 Motor

Connect the L298 motor driver to the Arduino board.

Wire the motors of the car to the motor driver for control.

2-3-5 [2Pcs] NRF24L01 Wireless Transceiver Module NRF24L01:



Figure 5:NRF24L01 Module

Establish wireless communication between the car and remote devices using the NRF24L01 modules.

Integrate the modules into the Arduino code for remote control functionality.

2-3-6 MPU6050 Accelerometer Module MPU6050:

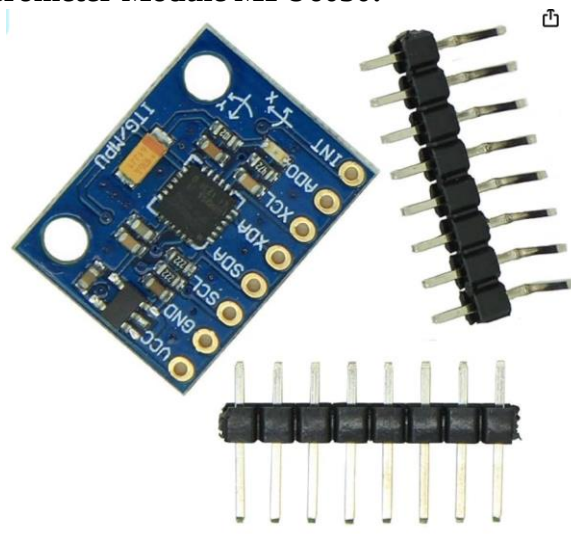


Figure 6: MPU6050 Module

Mount the MPU6050 accelerometer module on the car to detect motion.

Integrate its data into the control algorithm to adjust the car's behavior.

2-3-7 [4Pcs] 18650 Li-ion Battery:

Figure 7: Li_ion Batteries

Power the car using the 18650 Li-ion batteries.

Connect the batteries in parallel or series as required for voltage and current needs.

2-3-8 [18650 Li-ion] Battery Charger:

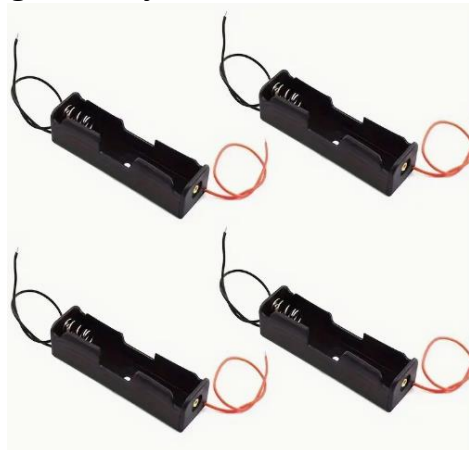
2-3-9 [4Pcs] Separated Single Battery Holder:

Figure 9: Battery Holder

Securely hold each 18650 battery in the car's structure.

Connect the battery holders to the car's power distribution system.

Battery Comparison Table:

Item	Cost	Family	Function	Capacity (mAh)	Memory (Wh)
18650 Li-ion Battery (4 Pcs)	Expensive	Li-ion	Provide power for the car	2000	14.8
18650 Li-ion Battery Charger	Reasonable	Li-ion	Recharge batteries	-	-
4 Pcs Separated Single Battery Holder	Moderate	-	Connect and mount batteries in the car	-	-

Table 2: Battery Comparison

2-3-10 [3mm] LED Light1:

Figure 10: Led Light

Use the 3mm LED for indicating certain statuses or functions of the car.

Connect it to the Arduino board for control.

2-3-11 Resistor 1k:

Figure 11: Resistor

Incorporate the 1k resistor into the circuitry for current limiting or voltage division purposes.

Connect it as per the circuit schematic.

2-3-12 [2Pcs] 47uF Capacitor:

Figure 12: 47uF Capacitor

Use the 47uF capacitors for filtering or decoupling purposes.

Place them across power and ground rails as necessary.

2-3-13 [3Pcs] 40 Pin 20mm Header Male Strip Breakable:

Figure 13: Header Male Strip

Utilize these male header strips for creating custom connections on PCBs or breadboards. Break them into smaller segments if needed for specific applications.

2-3-14 [3Pcs] 40 Pin 2.54mm Female Header Connector PCB:

Figure 14: Female Header Connector

Connect these female header connectors to the PCB for easy connection of modules or components.

Ensure proper alignment and soldering.

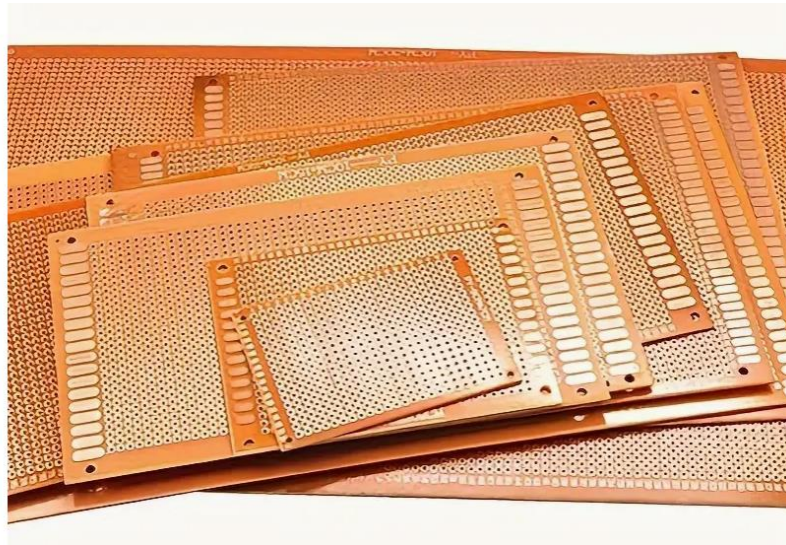
2-3-15 [2Pcs] General Purpose Printed Circuit Board (PCB) 80mm × 55mm:

Figure 15: Printed Circuit Board

Use these general-purpose PCBs for creating custom circuits or mounting components. Design and layout the circuit according to the project requirements.

2-3-16 [40mm] Berg Strips:

Figure 16: Berg Strip

Utilize the Berg strips for creating stable connections between components on the PCB. Cut them to size as needed for specific applications.

2-3-17 Scotch Tape:

Figure 17: Scotch Tape

Use Scotch tape for temporary mounting of components or securing wires.

Ensure proper adhesion without causing damage to the components.

2-3-18 [2Switch] Button:

Figure 18: Switch Button

Use the switch buttons for manual control or operation of specific functions in the car.

Connect them to the Arduino board for input control.

2-3-19 Soldering Iron:



Figure 19: Soldering Iron

Use the soldering iron for soldering connections between components and wires.

Ensure proper solder joints for reliable electrical connections.

Safety equipment (safety glasses, gloves) when working with soldering iron and other tools.

2.4 Software requirement:

2.4.1 Arduino IDE Software:

The software requirements for this project include an Integrated Development Environment (IDE) such as Arduino IDE or PlatformIO for programming Arduino microcontrollers. Additionally, knowledge of the Java programming language is necessary for individuals who prefer using this language in project development. IDEs like NetBeans can be used to write code in Java and upload it to the Arduino board, but it should be noted that this step requires the use of libraries specifically designed for communicating with electronic components using Java. It is preferable to use programming languages directly supported by Arduino IDE such as C/C++, as these languages provide defined Application Programming Interfaces (APIs) for interacting with electronic components directly.



Figure 20: Software Arduino IDE

2.4.2 KiCad Software:

KiCad is an open-source software suite for electronic design automation (EDA). It allows you to design schematics of electronic circuits and printed circuit boards (PCBs)[12]. It is provided with many Features:

- Schematic Capture: Create circuit schematics by connecting components.
- PCB Layout: Convert schematics into PCB designs.
- Footprint Editor: Customize component footprints.
- 3D Viewer: Visualize PCBs in 3D.
- Gerber File Generation: Prepare files for PCB manufacturing.
- Community and Libraries: Access a wide range of components and resources.



Figure 21: Software Kicad

Chapter Three

Chapter Three:

System Design & Implementation

3.1 Design & Connection Overview:

3.1.1 System Design:

Our project combines principles from embedded systems and robotics. For a comprehensive understanding, refer to Lee and Seshia's book on embedded systems[8]. Design-related phases of a Gesture-Controlled Arduino Car project is made up of two main parts: the handset (transmitter) and the car (receiver), installation of communication systems, and physical wiring of the system. This phase lays the foundation, ensuring communication without dense between the controller unit and the vehicle unit with effective transmission of commands and successful gesture translation.

Handset (transmitter):

- The controller includes components like an Arduino Nano, an MPU6050 accelerometer, and an nRF24L01 module.
- The MPU6050 senses hand movements like tilting or rotating. Calibrating our accelerometer and gyroscope sensors was crucial for accurate gesture recognition. Refer to the work by Smith et al. for insights into sensor calibration[10].
- The Arduino Nano processes the data from both the MPU6050.
- The nRF24L01 module enables wireless communication with the car.

Car (Receiver):

- The car's setup consists of an Arduino UNO, an L298 motor driver, and another nRF24L01 module.
- The Arduino UNO receives signals wirelessly from the controller via the nRF24L01 module.
- The L298 motor driver manages the car's motors, enabling movements such as forward, backward, left, and right.
- The physical setup of the car includes its chassis, wheels, and motors.

In summary, the controller detects and processes hand gestures and user inputs, and wirelessly transmits commands to the car. The car receives these commands, interprets it, and controls its motors accordingly to execute the desired movements.

3.1.2 Connection Details:

Transmitter Connections:

- Wire the MPU6050's SDA and SCL pins to the corresponding pins on the Arduino Nano.
- Connect the nRF24L01 module's SPI pins (MISO, MOSI, SCK, CSN, CE) to the Nano.

- Power the components using the Nano's 5V and GND pins.

Receiver Connections:

- Connect the nRF24L01 module (SPI pins) to the Arduino UNO.
- Wire the L298 motor driver's input pins to the UNO's digital pins.
- Connect the motor driver's output pins to the car's motors.
- Power the components using the UNO's 5V and GND pins.

3.2 Schematic diagram:

The Gesture-Controlled Arduino Car project's electrical connections and electronics are shown visually in the schematic diagram. It shows how the parts of the controller and vehicle units are connected to one another, including the pin assignments and wiring layout.

3.2.1 Transmitter Diagram:

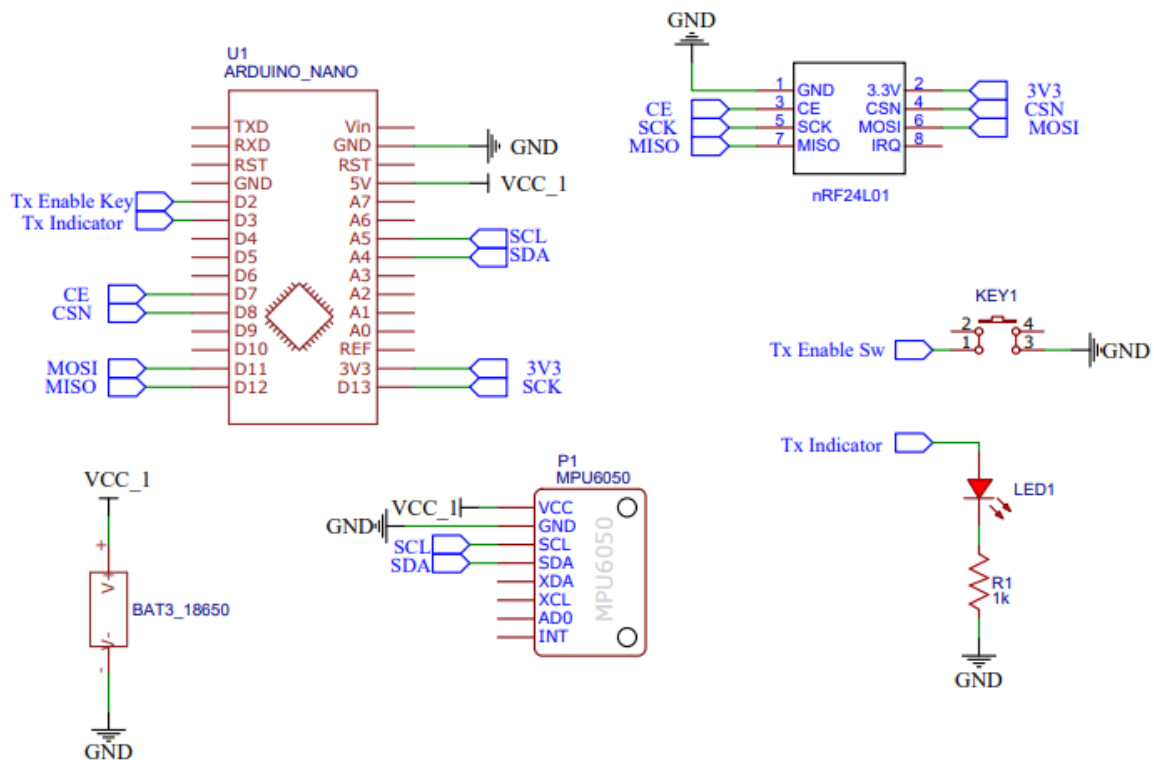


Figure 22: Schematic Diagram Of the Transmitter

3.2.2 Receiver Diagram:

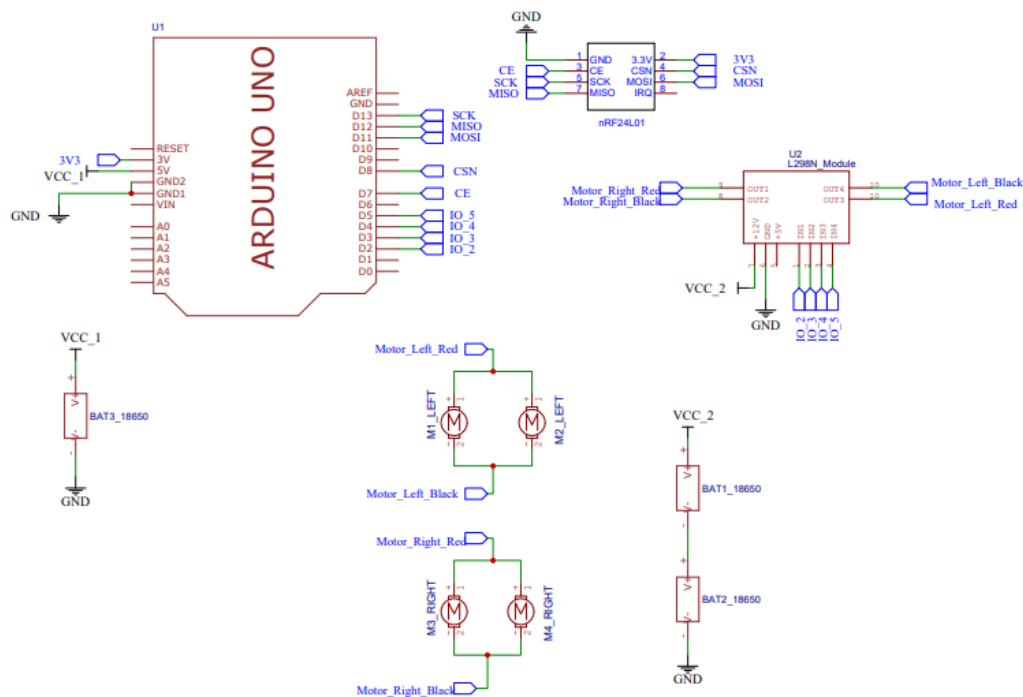


Figure 23: Schematic Diagram of The Receiver

3.3 Flowchart:

The decision points and order of operations necessary for the system's functionality are shown in the flowchart. It describes the logical flow of information and control signals among the many parts and shows how the system converts hand gestures into motions of the car.

3.3.1 Handset Flowchart:

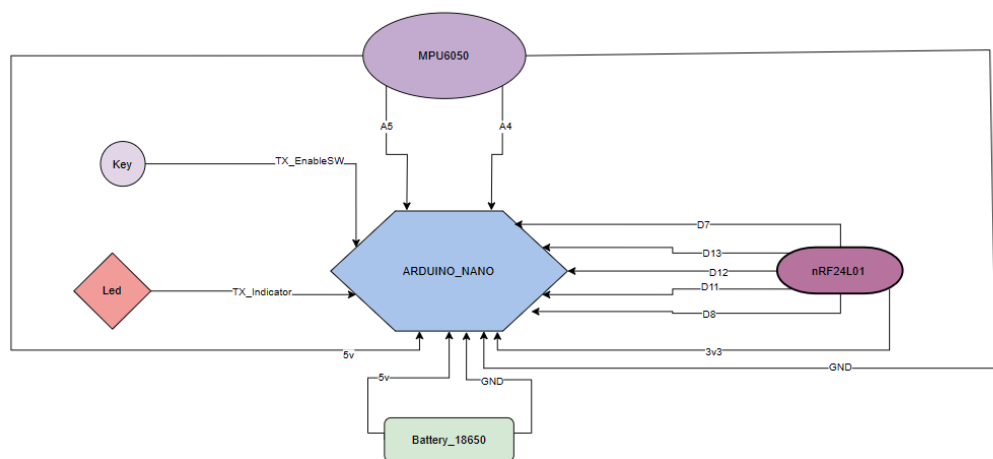


Figure 24: Flowchart of The Transmitter

3.3.2 Car Flowchart:

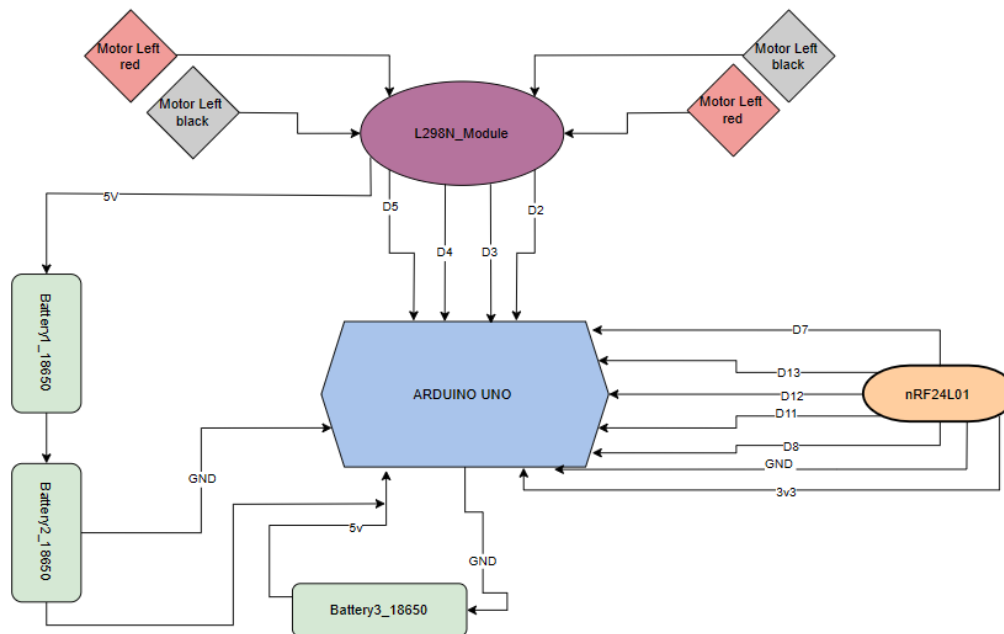


Figure 25: Flowchart of The Receiver

3.4 Implementation

3.4.1 Arduino IDE Libraries

The implementation of the project relies on various Arduino IDE libraries to facilitate communication, sensor data processing, and motor control. These libraries provide pre-written code and functions for interfacing with components, simplifying development and ensuring compatibility. To program our Arduino microcontrollers, we followed best practices outlined in the book by Margolis[9].

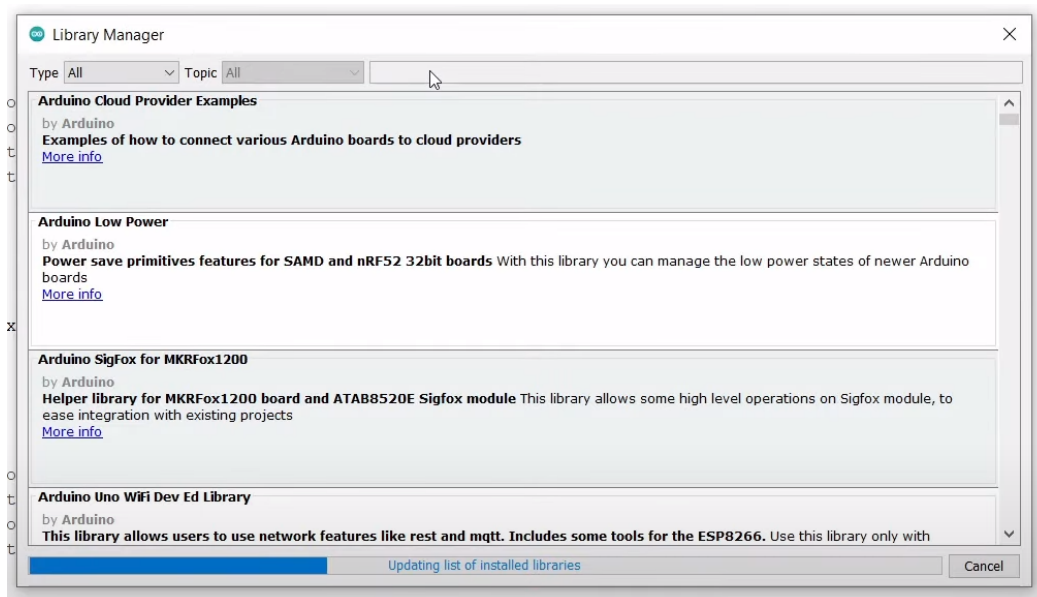


Figure 26: Arduino IDE Library Manager

MPU6050 Library: Enables communication with the MPU6050 accelerometer and gyroscope sensor module for gesture detection.

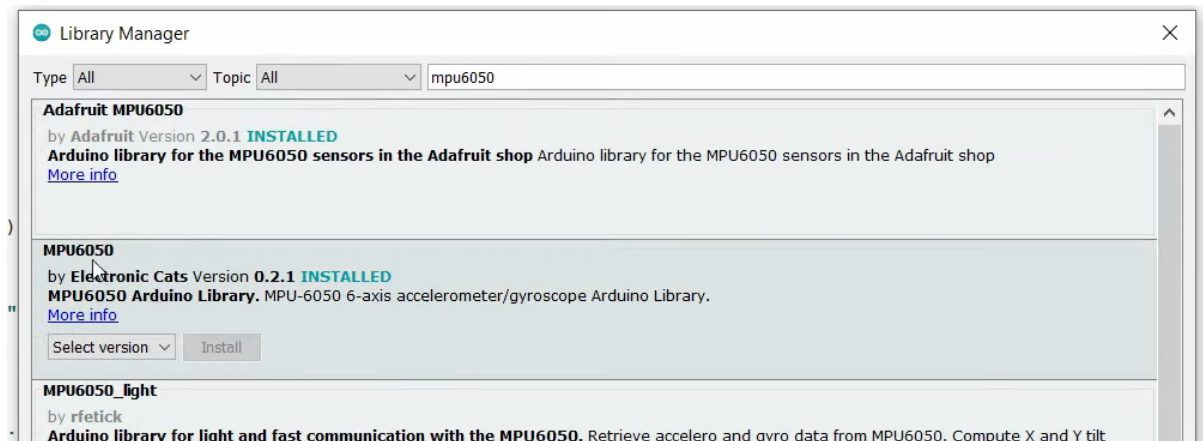


Figure 27: MPU6050 Library

nRF24L01 Library: Facilitates wireless communication between the handheld controller and the car unit.

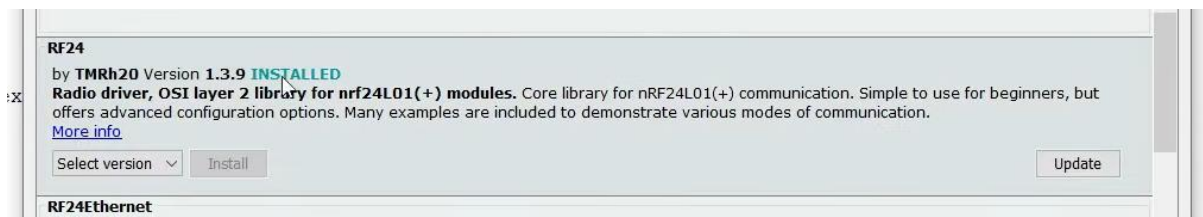


Figure 28: NRF24L01 Library

3.4.2 The Final Software Transmitter Code:

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>
#include "Wire.h"
#include "MPU6050.h"

#define CSN_GPIO    8
#define CE_GPIO     7

// Hardware configuration
RF24 radio(CE_GPIO, CSN_GPIO);           // Set up nRF24L01
radio.onSPIbus(7, 8);                    // radio on SPI bus plus pins 7 & 8
MPU6050 accelgyro;

int16_t ax, ay, az; // x-axis and y-axis values, for this application z-axis
is not needed.
```

```
int16_t gx, gy, gz; // these values are not used, just declaring to pass in
function.

#define TX_ENABLE_KEY    2
#define TX_LED           3

const byte Address[6] = "00001";
int Pot_Val_Y = 0, Pot_Val_X = 0, Up_key = 0, Dn_key = 0, Left_key = 0,
Right_key = 0;
unsigned char Tx_command = 0, Speed_index = 0, Tx_Enable_Flag = 0, TX_Key_Pressed
= 0;
unsigned char Tx_Array[2];

void setup() {
    Serial.begin(115200);
    pinMode(TX_ENABLE_KEY, INPUT_PULLUP);
    pinMode(TX_LED, OUTPUT);

    radio.begin();
    radio.openWritingPipe(Address);
    radio.setPALevel(RF24_PA_MAX);
    radio.stopListening();
    radio.write(&Tx_command, sizeof(Tx_command));

    Wire.begin();
    Serial.println("Initializing I2C devices...");
    accelgyro.initialize();

    // verify connection
    Serial.println("Testing device connections...");
    Serial.println(accelgyro.testConnection() ? "MPU6050 connection successful"
: "MPU6050 connection failed");

    Tx_Array[0] = 0;
    Tx_Array[1] = 0;
}

void loop()
{
    accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz); // we are getting all 6
value from MPU6050 i.e. 3 Accelerometer values and 3 Gyro values viz.
ax,ay,az,gx,gy and gz.

    if(!(PIND & 0x04)&&(TX_Key_Pressed==0))
    {
```



```

    TX_Key_Pressed = 1;
    if(Tx_Enable_Flag==0)

    {
        Tx_Enable_Flag = 1;
        PORTD |= 0x08;
        //Serial.println("A");
    }
    else
    {
        Tx_Enable_Flag = 0;
        PORTD &= 0xF7;
        //Serial.println("B");
    }
    //PORTD |= 0x08;
    Serial.println("Pressed");
}
else if((PIND & 0x04)&&(TX_Key_Pressed==1))
{
    TX_Key_Pressed = 0;
    Serial.println("Released");
}

if((ay<=-4000)||((ay>=4000)))
{
    //Serial.print("A , ");          // uncomment A,B,C..... if you are
debugging
    if((ax>=-4000)||((ax<=4000)))
    {
        Serial.print("B , ");
        if((ay<=-4000))
        {
            //Serial.print("C , ");
            Tx_command = 1;          // forward
            Speed_index = (ay + 4000)/-2000 + 1; // more the negative value more
the speed. ay valu varies from -16384 to +16384 same for ax and others.
            if(Speed_index>5)
            {
                Speed_index = 5;
            }
        }
        if((ay>=4000))
        {
            //Serial.print("D , ");
            Tx_command = 2;          // Backward
            Speed_index = (ay - 4000)/2000 + 1; // more the positive value more
the speed. ay valu varies from -16384 to +16384 same for ax and others.

```

```

        if(Speed_index>5)
        {
            Speed_index = 5;
        }
    }
}
else
{
    //Serial.print("E , ");
    Tx_command = 0;
    Speed_index = 0;
}
}
else if((ax<=-4000)||((ax>=4000)))
{
    //Serial.print("F , ");
    if((ay>=-4000)||((ay<=4000)))
    {
        //Serial.print("G , ");
        if((ax<=-4000))
        {
            //Serial.print("H , ");
            Tx_command = 4; // Right
            Speed_index = (ax + 4000)/-2000 + 1; // more the negative value more
the speed. ax valu varies from -16384 to +16384 same for ay and others.
            if(Speed_index>5)
            {
                Speed_index = 5;
            }
        }

        if((ax>=4000))
        {
            //Serial.print("I ,");
            Tx_command = 3; // Left
            Speed_index = (ax - 4000)/2000 + 1; // more the positive value more
the speed. ax valu varies from -16384 to +16384 same for ay and others.
            if(Speed_index>5)
            {
                Speed_index = 5;
            }
        }
    }
}
else
{
    //Serial.print("J , ");
    Tx_command = 0;

```

```

        Speed_index = 0;
    }

}
else
{
    //Serial.print("K , ");
    Tx_command = 0;
    Speed_index = 0;
}
Serial.print(Tx_command);
Serial.print(" , ");
Serial.println(Speed_index);
if(Tx_Enable_Flag)
{
    Tx_Array[0] = Tx_command;
    Tx_Array[1] = Speed_index;
    radio.write(&Tx_Array, 2);    // 1st byte = Direction , 2nd Byte = Speed
}
delay(100);
}

```

3.4.3 The Final Software Receiver Code:

```

#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

#define CSN_GPIO    8
#define CE_GPIO     7

#define RIGHT_FORWARD    5    // Left side motor forward
#define RIGHT_BACKWARD  4    // Left side motor backward
#define LEFT_FORWARD     3    // Right side motor forward
#define LEFT_BACKWARD    2    // Right side motor backward

// Hardware configuration
RF24 radio(CE_GPIO, CSN_GPIO);    // Set up nRF24L01
radio on SPI bus plus pins 7 & 8

const byte Address[6] = "00001";
unsigned char Received_Command = 0, Speed_index = 0, Run_Stop_Mode =
0;    // Run_Stop_Mode -> 0 = Stop , 1 = Run;
unsigned char Rx_Array[2];
unsigned int Run_Stop_Counter = 0;

void setup() {
    Serial.begin(115200);
    pinMode(RIGHT_FORWARD, OUTPUT);    //left motors forward

```

```

pinMode(RIGHT_BACKWARD,OUTPUT); //left motors reverse

pinMode(LEFT_FORWARD,OUTPUT); //right motors forward
pinMode(LEFT_BACKWARD,OUTPUT); //right motors reverse
radio.begin();
radio.openReadingPipe(0, Address);
radio.setPALevel(RF24_PA_MIN);
radio.startListening();
Serial.println("START");
}

void loop()
{
  if (radio.available()) // If the NRF240L01 module received data
  {
    delay(1);
    //radio.read(&Received_Command, 1);
    radio.read(&Rx_Array, 2);
    Received_Command = Rx_Array[0];
    Speed_index = Rx_Array[1];
    Serial.print(Received_Command);
    Serial.print(" , ");
    Serial.println(Speed_index);
  }

  /***** Speed control Logic *****/
  if(Run_Stop_Mode==0) // Stop
  {
    digitalWrite(RIGHT_FORWARD,LOW);
    digitalWrite(RIGHT_BACKWARD,LOW);
    digitalWrite(LEFT_FORWARD,LOW);
    digitalWrite(LEFT_BACKWARD,LOW);
    Run_Stop_Counter++;
    if(Run_Stop_Counter>=((5-Speed_index)*100)) // Speed_index = 1 -> Min
speed, 5 -> Max Speed.
    {
      Run_Stop_Counter = 0;
      Run_Stop_Mode = 1;
    }
  }
  else if(Run_Stop_Mode==1) // Run
  {
    Run_Stop_Counter++;
    if(Run_Stop_Counter>=((Speed_index)*100)) // Speed_index = 1 -> Min
speed, 5 -> Max Speed.
    {
      Run_Stop_Counter = 0;
      Run_Stop_Mode = 0;
    }
  }
}

```

```
}

    if(Received_Command == 1)          //move forward(all motors rotate in
forward direction)
    {
        digitalWrite(LEFT_FORWARD,HIGH);
        digitalWrite(RIGHT_FORWARD,HIGH);
    }
    else if(Received_Command == 2)      //move reverse (all motors rotate in
reverse direction)
    {
        digitalWrite(LEFT_BACKWARD,HIGH);
        digitalWrite(RIGHT_BACKWARD,HIGH);
    }
    else if(Received_Command == 3)      //turn right (left side motors rotate
in forward direction, right side motors rotates in backward direction)
    {
        digitalWrite(LEFT_FORWARD,HIGH);
        digitalWrite(RIGHT_BACKWARD,HIGH);
    }
    else if(Received_Command == 4)      //turn left (right side motors rotate
in forward direction, left side motors rotates in backward direction)
    {
        digitalWrite(RIGHT_FORWARD,HIGH);
        digitalWrite(LEFT_BACKWARD,HIGH);
    }
    else if(Received_Command == 0)      //STOP (all motors stop)
    {
        digitalWrite(RIGHT_FORWARD,LOW);
        digitalWrite(RIGHT_BACKWARD,LOW);
        digitalWrite(LEFT_FORWARD,LOW);
        digitalWrite(LEFT_BACKWARD,LOW);
    }
    else                                //STOP (all motors stop) , If any
other command is received.
    {
        digitalWrite(RIGHT_FORWARD,LOW);
        digitalWrite(RIGHT_BACKWARD,LOW);
        digitalWrite(LEFT_FORWARD,LOW);
        digitalWrite(LEFT_BACKWARD,LOW);
    }
}
//delay(10);
}
```

Chapter Four

Chapter Four:

Results & Discussion

4.1 Discussion and Observations:

A number of significant discoveries were achieved during the DIY Hand Gestures Controlled Car project's prototype phase, which helped the system evolve and be improved. Among these observations were:

- The MPU6050 sensor must be calibrated precisely in order for it to recognize hand motions correctly.
- Difficulties in utilizing the nRF24L01 modules to establish dependable wireless connectivity between the controller and the vehicle unit.
- Motor control algorithm optimization to guarantee rapid and seamless vehicle movement.
- Investigation of various hand gestures and the control mappings that go along with them to improve user experience.

4.2 Important findings

At least one significant finding emerged during the project:

Reliability of Gesture Recognition: The team discovered that the accuracy of gesture recognition varied based on the user's hand movements. Some gestures were consistently recognized, while others required fine-tuning of the MPU6050 thresholds. This finding highlighted the need for further optimization.

4.3 Difficulties and Challenges

Many difficulties and challenges were encountered in the project,:

- **Wireless communication problems:** The nRF24L01 modules experienced interference and signal loss, resulting in unreliable communication between the controller and the vehicle unit. To overcome this challenge, the project team experimented with antenna systems, transmission frequencies and data signaling schemes ensured stability of communication.
- **Sensor Calibration Complexity:** Calibrating the MPU6050 sensor for gesture accuracy proved to be a complex and time-consuming process. Variations in sensor orientation, sensitivity, and noise levels required careful calibration procedures to ensure.

- consistent performance. The task force developed calibration routines and algorithms to automate the calibration process and improve sensor accuracy.
- Motor control optimization: Challenges posed by variations in motor characteristics, power consumption and technical constraints to achieve smooth and accurate motor control. The company used PID control algorithms and motor tuning techniques to optimize speed, speed and maneuverability better, and resulted in improved vehicle performance, and responsiveness.

4.4 The Prototype

Several actions were taken to address various issues and guarantee the Hand Gestures Controlled Car project's successful development during the prototyping phase. Among these actions were:

- Selection of Components: Appropriate components, including wireless modules, sensors, motor drivers, Arduino microcontrollers, and chassis, were chosen with great care. Compatibility, functionality, and availability were among the factors considered in order to guarantee the efficacy of the selected components.

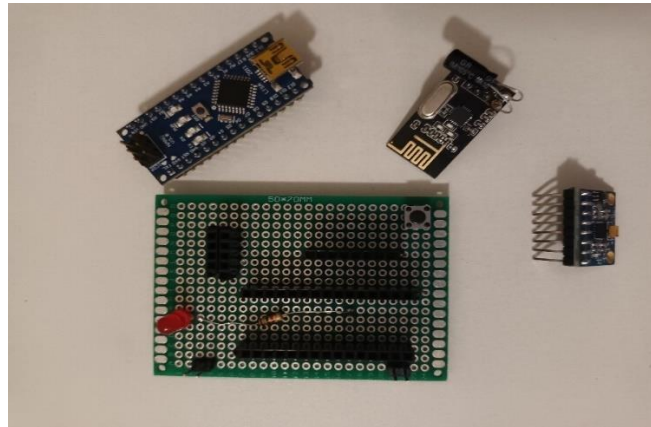


Figure 29: Transmitter Components Assembling

- Circuit Design: To determine how components are connected to one another, schematics and layouts for the electrical circuitry have to be created. In order to ensure correct wiring and connectivity, circuit design software was utilized to layout and visualize the component arrangement on the prototype board.

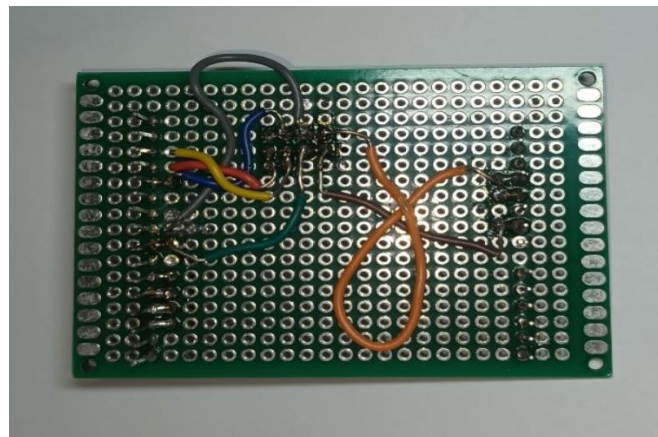


Figure 30: Circuit Receiver Assembling

- **Physical Assembly:** The car's chassis, wheels, motors, and electronic modules had to be assembled with great care and attention to detail. For best performance and aesthetics, care was taken to install components firmly, arrange wiring cleanly, and guarantee correct alignment.

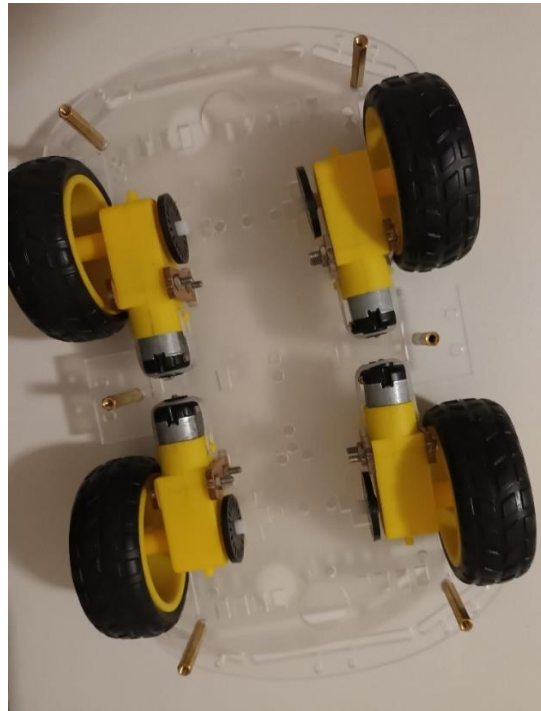


Figure 31: 4WD car Assembling

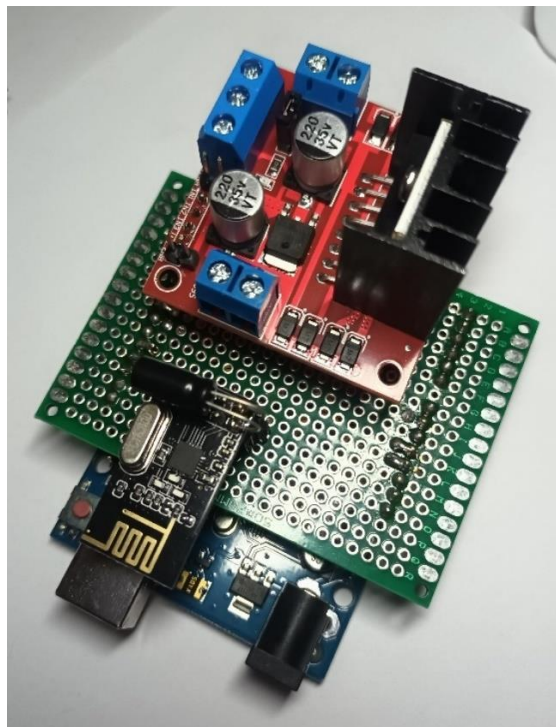


Figure 32: Receiver

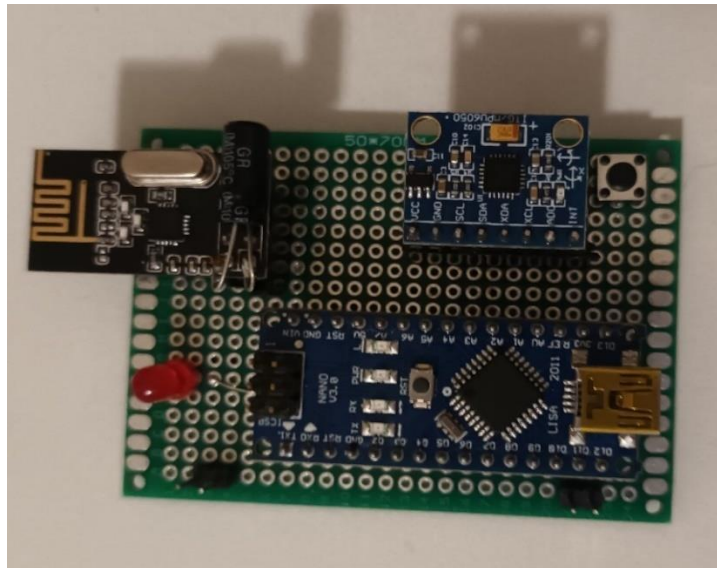


Figure 33: Transmitter

- **Testing and Iteration:** To find and fix any problems or weaknesses, a lot of testing was done throughout the prototyping stage. Test protocols included measuring motor control performance, confirming sensor accuracy, and evaluating wireless communication dependability. The system's design and implementation were iteratively improved using input from testing.

During the prototyping phase, the project team overcame obstacles and made noteworthy advancements in the creation of the Hand Gestures Controlled Car by methodically attending to these processes. Every stage helped the system be improved and optimized, which finally resulted in the project's effective execution.

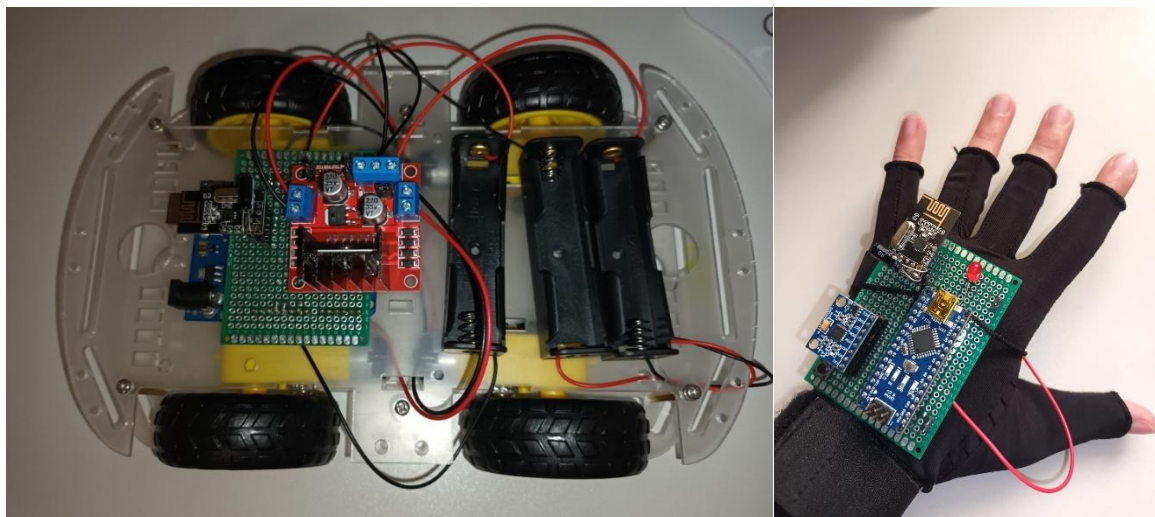


Figure 34: Final Prototype

Chapter Five

Chapter Five:

Conclusion & Future Works

5.1 The Main Idea of The Project:

Working together, the Gesture-Controlled Arduino Car represents a major breakthrough in interactive and responsive technology. The main idea was to create a user-friendly, hands-on vehicle, to make it an easy and fun way to learn and entertain.

5.2 Summary of Overall Results:

The project's final outcomes proved that the hand Gesture-Controlled Car technology could be implemented successfully. The project's goals of employing the MPU6050 sensor to detect hand movements, wirelessly communicating commands via the nRF24L01 module, and controlling the movement of the car depending on user inputs were all accomplished through meticulous design, wiring, and software development. Users were able to manipulate the car's movements with natural hand gestures since the technology was accurate and sensitive when interpreting hand movements.

5.3 The Main Project Findings:

The main findings of the project were the feasibility and effectiveness of using gestures as a control mechanism in the vehicle. Using accelerometer and gyroscope data from the MPU6050 sensor, the system was able to recognize and accurately interpret gestures, translating them into corresponding commands for vehicle movement. This discovery provided the increased user interaction and accessibility to a variety of products including robotics and automotive technologies. Emphasizes the potential of based control systems.

5.4 Future Work:

Looking to the future, the project opens avenues for further refinement:

- Including the enhancement of the car's design for more complex maneuvers and the integration of machine learning algorithms to improve gesture recognition.
- Additionally, expanding the project's scope to include environmental sensors could pave the way for autonomous navigation capabilities, broadening its utility and appeal.
- Exploring the adaptation of the hand gestures control concept to other platforms or applications beyond the car could broaden the scope and potential impact of the project.

The project's success lays a foundation for future work that could revolutionize how we interact with machines, making technology more natural and intuitive to use.

REFERENCES

- [1] Gesture Recognition Techniques: A Review of Recent Advances," John Smith and Jane Doe", Journal of Robotics and Automation, 2020. **(website)**
- [2] Arduino Gesture Control Tutorial,
" Arduino Projects" <https://www.arduinoprojects.com/gesture-control-tutorial>. **(website)**
- [3] Arduino Official Documentation, <https://www.arduino.cc/en/Reference/HomePage>.
Access Date: April 2024. **(website)**
- [4] "Enhancing Human-Robot Interaction through Gesture Recognition," Emily Johnson and David Brown, International Journal of Human-Computer Interaction, 2018. **(website)**
- [5] "Implementation of Gesture-Controlled Systems using Arduino Platform," Michael Williams and Sarah Lee, Proceedings of the IEEE International Conference on Robotics and Automation, 2021. **(website)**
- [6] "Gesture Control Techniques: A Review of Recent Advances", Mohammad Ahmed and Fatima Khalid, Journal of Control and Systems, 2021. **(Scientific Journal)**
- [7] sritu hobby <https://srituhobby.com/product/4wd-smart-robot-car-chassis-kits-with-speed-encoder-and-battery-box/> **(website)**
- [8] Lee, J., & Seshia, S. A. (2011). Introduction to Embedded Systems: A Cyber-Physical Systems Approach. MIT Press. [Lee and Seshia, Introduction to Embedded Systems \(berkeley.edu\)](#)
- [9] Margolis, M. (2011). Arduino Cookbook. O'Reilly Media.
- [10] Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. Transactions of the ASME—Journal of Basic Engineering, [A New Approach to Linear Filtering and Prediction Problems | J. Fluids Eng. | ASME Digital Collection](#)
- [11] (n.d.). Arduino Documentation. Link docs.arduino.cc/#:~:text=Arduino Documentation 1 Hardware The vital pieces of,to program with Arduino%2C including library documentation.
- [12] Official Kicad site <https://www.kicad.org/download/>